
Workgroup:	SCHC Working Group				
Internet-Draft:	draft-ietf-schc-architecture				
Published:	4 July 2026				
Intended Status:	Informational				
Expires:	5 January 2027				
Authors:	A. Pelov	P. Thubert	A. Minaburo	Q. Lampin	M. Dumay
	<i>IMT Atlantique</i>		<i>Consultant</i>	<i>Orange</i>	<i>Orange</i>

Static Context Header Compression (SCHC) Architecture

Abstract

TODO Abstract

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 5 January 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction	3
2. Terminology	3
3. SCHC: Primer	5
3.1. SCHC Invocation: Network Service vs. Application Service	5
4. Architecture	6
4.1. Overview of a Basic Architecture	6
4.1.1. Basic SCHC Architecture	6
4.1.2. Multiple Instances Architecture	7
4.1.3. SCHC configuration provisioning architecture	8
4.2. Focus on core components	10
4.2.1. Instance	10
4.2.2. Endpoint	10
4.2.3. Session	11
4.2.4. Domain	12
4.2.5. Datagram Format	13
5. Deployment models	14
5.1. LPWAN deployment	14
5.2. 6Lo deployments	15
6. Operational considerations	17
7. Security Considerations	18
8. IANA Considerations	18
9. References	18
9.1. Normative References	18
9.2. Informative References	19
Appendix A. Acknowledgments {numbered="false"}	20
Authors' Addresses	20

1. Introduction

The SCHC Working Group has developed the [\[RFC8724\]](#) SCHC protocol for Low-Power Wide-Area (LPWA) networks, providing efficient header compression and fragmentation mechanisms.

Requirements Language

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP14 [\[RFC2119\]](#) [\[RFC8174\]](#) when, and only when, they appear in all capitals, as shown here.

Check the use of normative keywords. (ex.: "Since [\[RFC8724\]](#) requires X, an architecture compliant with this document MUST ...") Be very clear about what constitutes a "requirement on implementations" versus a "recommendation on deployments." Ensure that you do not introduce requirements that would contradict RFC 8724 or existing profiles (SCHC over LoRaWAN, NB-IoT, etc.).

2. Terminology

This section defines terminology and abbreviations used in this document. In the following, terms are assumed to be defined in the context of the SCHC ecosystem, unless specified otherwise, *e.g.* Endpoint refers to a SCHC Endpoint, Instance refers to a SCHC Instance, and so on.

SCHC: A Generic Framework, as defined in [\[RFC8724\]](#), that performs compression/decompression and, optionally, fragmentation/reassembly of protocol headers, based on a Static Context shared between two or more Endpoints. The SCHC acronym is pronounced like "sheek" in English (or "chic" in French).

Endpoint: A logical entity that provides SCHC functionality by hosting the SCHC processing code, rather than a physical device. Multiple SCHC Endpoints can operate on the same physical equipment, for example to serve different Domains, tenants, strata.

Instance: A logical component of an Endpoint that executes the actual SCHC operations, *e.g.* compressing and decompressing headers, fragmenting and reassembling packets. Multiple Instances can coexist on the same Endpoint but each Instance operates independently, with its own Context and Configuration.

Rule: A structured set of header fields and matching conditions used by SCHC to process packets in accordance with specified actions.

Set of Rules (SoR): A collection of Rule entries that define how specific header fields are processed by an Instance.

Context: A SoR together with metadata, shared by two or more Instances. Metadata may, for example, refer to a data model or a parser compatible with the rule format.

Instance Configuration: A set of configurations specific to an Instance that define how SCHC operations are performed, e.g. role of the Instance, matching policy, dispatcher configuration, supported SCHC features.

Session: A communication session between two or more Instances that share a common Context for SCHC operations.

Set of Variables (SoV): Runtime parameters and session variables, such as fragmentation-related timers, retransmission counters, state flags, and other per-session values that may change during operation.

Dispatcher: Entity that uses the Discriminator to route a Datagram to the appropriate Instance. The Dispatcher can be integrated into the network stack or be a separate component.

Discriminator: Information element derived from the Carrier Layer that enables the selection of the correct Instance. The Discriminator can be an **explicit** field in the Datagram or frame (for example, a 6LoWPAN Dispatch byte, a VOICI Session ID).

Carrier Layer: The layer adjacent to and below the Stratum's lower boundary, from which the Discriminator is derived. The Carrier Layer is outside the Stratum and not addressable by the Instance's Rules. Examples include IEEE 802.15.4, LPWAN links, PPP frames.

Parser: A software tool or component that dissects and analyzes network packets, to extract header fields such as source and destination addresses, port numbers, and payload data.

Domain: A logical grouping of Instances that share a common set of Contexts for SCHC operations.

Stratum: A background concept that defines the scope of headers an Instance addresses. It is expressed as a contiguous range [**lower boundary, upper boundary**] of protocol layers, where both boundaries are inclusive. If lower boundary equals upper boundary, the Stratum spans a single layer. Headers originating from above the upper boundary are unaffected by the Instance. The term "Stratum" originates from the idea of layers in a network stack but is applicable beyond the network stack, for instance to application-layer protocol headers.

Datagram: The unit exchanged between SCHC instances. A Datagram consists of a Rule Identifier (RuleID) and the result of the SCHC operation (if non-empty), such as a compression residue or a packet fragment. It ends with a Payload taken from the original packet (if any).

Domain Manager: A logical component that manages the Domain, including context synchronization and configuration distribution.

Instance Manager: A logical component that manages the lifecycle and configuration of Instances within an Endpoint. It is responsible for creating, updating, and deleting Instances as needed, synchronizing Contexts, and managing Instance Configurations.

Endpoint Manager:

Context Repository: A logical component that stores and manages the Contexts used by its Domain.

C/D: SCHC function that performs the Compression and Decompression of headers.

F/R: SCHC function that performs the Fragmentation and Reassembly of headers.

MO: Matching Operator

CDA: Compression/Decompression Action

3. SCHC: Primer

SCHC is primarily a compression mechanism for structured datagrams comprising headers. It reduces the size of these datagrams by exploiting predictable patterns found in application data and network header structures, e.g., static field values, network prefixes in addresses, known port numbers.

Instead of transmitting headers in full, the sender replaces known header fields with a compressed residue and an identifier -- a **RuleID** that instructs the receiver how to reconstruct the original headers. Both the sender and receiver share the **set of Rules** (SoR) needed for this process through a shared **Context**.

This shared Context may be static, for example pre-provisioned with the device firmware or installed dynamically. In the latter case, the logical groupings that share the same Context, and therefore require timely and coordinated updates, are called **Domains**.

Compression and decompression occur in SCHC **Instances** --- logical entities that: - Match datagram headers against Rules and compress them - Identify the Rule used by the compressor (via RuleID) and reconstruct the headers on decompression

Each Instance addresses a set of adjacent headers --- for example, the IPv6, UDP and CoAP headers in a network stack. This "slice" of the stack is called a **Stratum** and is defined by a lower boundary (the layer closest to the physical interface) and an upper boundary (the highest layer in the stack).

3.1. SCHC Invocation: Network Service vs. Application Service

SCHC can be used in two fundamentally different ways.

Case A: SCHC as a Network Service. The SCHC Instance acts on headers the stack itself needs to inspect and route. By itself, an Instance cannot intercept packets in the network stack, and SCHC is not a protocol residing in the stack. Because SCHC is not a protocol in the stack, an entity of the network stack, the **Dispatcher**, routes Datagrams to and from Instances.

On the compressor side, the Dispatcher intercepts **Native** Datagrams targeted by an Instance after the network protocols of the Stratum have finished processing the Datagram. The Dispatcher then dispatches the Native Datagram to the Instance which compresses the Datagram

to form the SCHC Datagram. The SCHC Datagram is then reinjected into the network stack at the adjacent layer below the Stratum -- the **Carrier** Layer. Eventually, the Dispatcher fills the Carrier **Discriminator**, a value of the Carrier header that indicates the Datagram is compressed with SCHC. The SCHC Datagram is then processed further by the network stack and sent to its recipient.

On the decompressor side, the SCHC Datagram is delivered by the network stack to the Dispatcher, eventually identified by the Carrier Discriminator. The Dispatcher delivers the SCHC Datagram to the appropriate Instance, which processes the SCHC Datagram to reconstruct the headers of the Stratum. Once reconstructed, the Dispatcher restore the Carrier Layer with the Native Datagram field values, for instance when a Carrier Discriminator is used.

The Datagram is further processed by the Network stack, as would the Native Datagram be processed.

The same core architectural concepts (Instance, Context, Rule, Stratum, Domain) apply in both cases. What differs is the routing and configuration mechanism: Carrier Layer, Discriminator and Dispatcher apply only to Case A.

For example, in the 6Lo scenario detailed below, the Instance addresses an IPv6/UDP/CoAP Stratum above the 802.15.4 L2 header. The Dispatcher is the stack routine that redirects frames bearing the SCHC Dispatch byte (0100 0100) to the correct Instance. The Dispatch byte is the **Discriminator** --- the information element the Dispatcher uses to route SCHC Datagrams.

The Discriminator usually sits in the adjacent layer below the Stratum lower boundary, outside the scope of addressable (compressible) headers for the SCHC instance.

Case B: SCHC as an Application Service. The Instance is invoked by the application's processing pipeline, compressing data that does not affect the network stack processing. The application calls the Instance as a library function, in an order the application dictates. No Dispatch, or Discriminator is needed. The Instance's Invocation Context is the application's processing stage.

4. Architecture

4.1. Overview of a Basic Architecture

4.1.1. Basic SCHC Architecture

[Figure 1](#) illustrates how messages are exchanged between applications running on two remote hosts using SCHC Compression/Decompression and optionally Fragmentation/Reassembly.

Each host features an **Endpoint** that implements SCHC functions. The effective Compression/Decompression or Fragmentation/Reassembly of datagrams is carried out by the **Instance**, a runtime entity.

How Compression/Decompression and Fragmentation/Reassembly is performed by each Instance is specified in two configuration elements: the **Context** which contains the configuration shared and identical between the two Instances and the **Instance Configuration** which is specific to each Instance.

Most notably, the Context contains the **Set of Rules** (SoR), which defines how datagrams are compressed and/or fragmented, and the Instance Configuration specifies which role each Instance assumes (Uplink/Downlink).

The two Instances that share the same Context form a **Session**, eventually identified by a **Session ID** which is then specified in the Context.

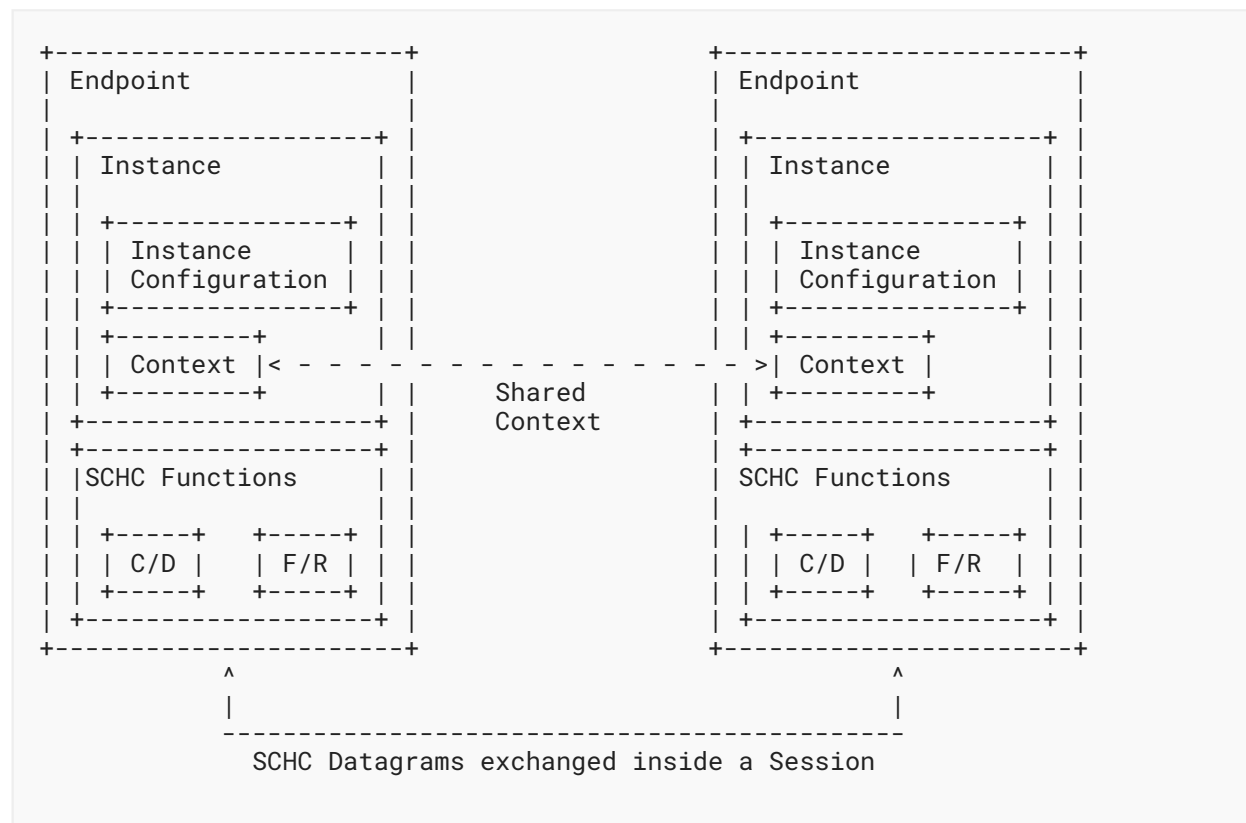


Figure 1: Overview of two simple Endpoints exchanging SCHC Datagrams

Important notice: having the same Context is not sufficient to guarantee the interoperability of SCHC operations between two Instances. The format of the data obtained from the Parser when processing the headers must be consistent on each Endpoint to allow the successful decompression. To ensure interoperability, the Context may specify which Parser to use to delineate the header fields, and/or which Data Model, such as the one defined in [RFC9363](#).

4.1.2. Multiple Instances Architecture

An Endpoint can host multiple Instances, each with its own Context and Instance Configuration.

When an Endpoint is supporting multiple Instances, the **Instance Manager** is responsible for managing the lifecycle and configuration of these Instances. Datagrams are routed to the appropriate Instance by the Dispatcher using the Discriminator and admission rules based on information provided in the Instance Configuration. The Dispatcher is a single point of decision for packet forwarding within the Endpoint.

In some deployments, the Discriminator is derived entirely from lower-layer context (e.g., a specific PPP link, an IPv6 address, or a UDP port). If external context is insufficient or unavailable, the Dispatcher may need an explicit Discriminator. For example, Datagrams can be encapsulated in a light transport protocol whose header contains a Session, Context, or Instance identifier, and can provide additional services such as integrity checking (CRC).

The following figure illustrates the main components of an Endpoint supporting multiple Instances and their interactions:

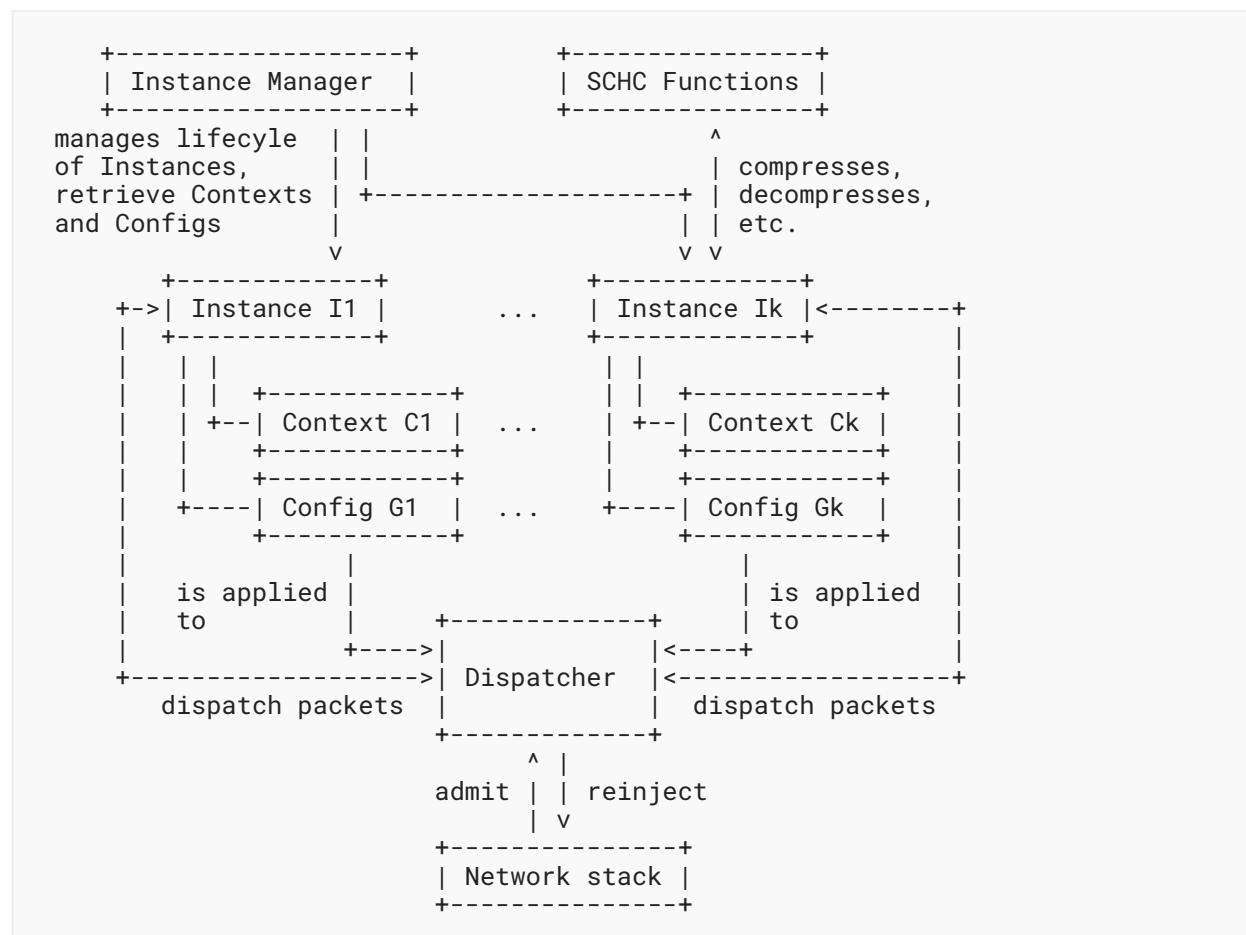


Figure 2: Overview of an Endpoints hosting multiple Instances

4.1.3. SCHC configuration provisioning architecture

Endpoints with Instances sharing a common Context may register to a **Domain**.

When Contexts are provisioned dynamically, a **Domain Manager** is responsible for this provisioning. within a Domain, the **Endpoint Manager** is in charge of the on-boarding of participating Endpoints. Contexts are stored in the **Context Repository** and the **Context Manager** is in charge of the synchronization of the instances' Contexts. Similarly, the **Instance Configuration Manager** is in charge of the deployment of the Instance-specific configurations.

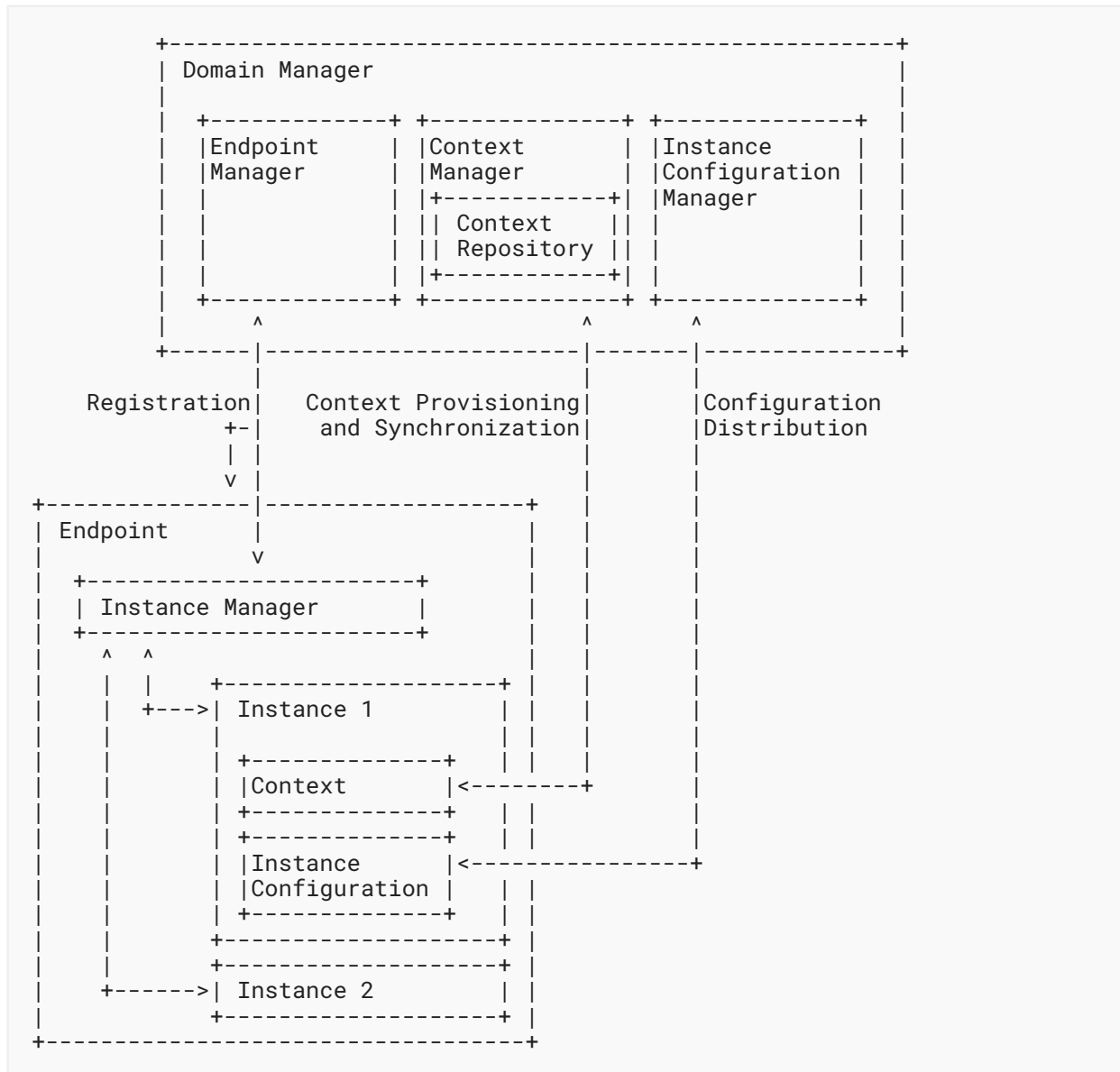


Figure 3: Overview of the functions of the Domain Manager

4.2. Focus on core components

4.2.1. Instance

An Instance is the fundamental component that runs a set of SCHC functionalities as defined in [RFC8724] hosted on an Endpoint. Its operation is defined by an Instance Configuration and a Context. An Endpoint MAY execute several Instances. Each Instance operates independently, with its own Context and Instance Configuration. Instance may execute dynamic Context update mechanisms and performance monitoring and reporting in complex scenarios.

4.2.1.1. Instance Configuration

The Instance Configuration specifies the local parameters of the Instance.

The Instance Configuration may indicate in a Manifest the set of required SCHC functionalities, such as:

- Header Compression and Decompression (C/D)
- Fragmentation and Reassembly (F/R)
- SCHClets (nodular subfunctions)

The Instance Configuration may also include the following parameters:

- Role of the Instance (e.g., Upside or Downside for asymmetric Rules).
- Matching policy (e.g., first-match, best-match, etc.) to apply when multiple rules match a packet.
- Packet interception criteria (e.g., Stratum - the protocol headers that the SCHC Rules in the associated Context can address, Filters based on specific values or characteristics of packets, etc.)
- Dispatch information (e.g., how to identify the Instance for incoming packets, how to route packets to the appropriate Instance, etc.).

4.2.1.2. Context

The Context contains operational information shared between two or more Instances.

For example, for Header Compression and Decompression (C/D) or Fragmentation and Reassembly (F/R), the Context defines the set of C/D and F/R Rules - or Set of Rules - describing the specific actions to be performed on the packets using the corresponding SCHC functionalities, and, optionally, the Parser and the Data Model to delineate and dissect the header fields.

4.2.2. Endpoint

A network entity providing SCHC functionalities, and hosting the Instances that consist of a specific execution of one or more of these aforementioned functionalities.

4.2.2.1. Header Compression and Decompression (C/D)

This component is responsible for compressing and decompressing headers using the SCHC framework, as described in [RFC8724]. It applies the rules defined in the Context.

The C/D engine MUST expose the following interface:

- `compress(buffer, context, config)`: Compresses the provided buffer using the Context and the Instance Configuration.
- `decompress(buffer, context, config)`: Decompresses the provided buffer using the Context and the Instance Configuration.

Internally, on compression, the C/D engine:

- delineates the fields using the Parser and/or Data Model provided in the Context;
- chooses the appropriate compression Rule among candidate Rules from the Context based on the matching policy defined in the Instance Configuration;
- applies the compression Rule to the fields of the header(s);
- generates the compressed SCHC Datagram. In [RFC8724], a packet whose header has been compressed is called a SCHC Packet.

On decompression, the C/D engine:

- identifies the appropriate decompression Rule based on the RuleID stored in the SCHC Packet;
- applies the decompression Rule to reconstruct the original header;
- reconstructs and returns the original packet from the decompressed header and payload.

4.2.2.2. Fragmentation and Reassembly (F/R)

This component is responsible for fragmenting larger packets into smaller fragments and reassembling them at the receiving end. It is optional feature but recommended for scenarios where packet sizes may exceed the maximum transmission unit (MTU) of the underlying network. In [RFC8724], the pieces of a SCHC Packet that has been fragmented are called SCHC Fragments.

4.2.2.3. SCHClets

A SCHClet is a self-contained unit within the SCHC framework that implements a specific SCHC function or a subset of SCHC operations. A SCHClet may implement aspects defined in [RFC8724] or functions from other related SCHC RFCs, and MAY be combined with other SCHClets within an Instance, as specified in the Manifest in the Instance Configuration.

4.2.3. Session

As illustrated in the figure below, the Session is a communication session between two or more Instances that share a common Context, i.e. they are part of the same Domain.

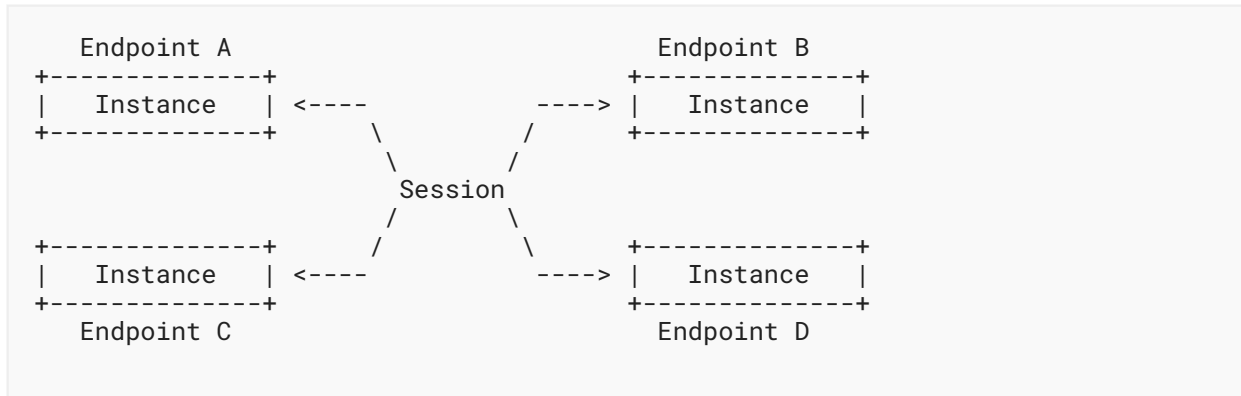


Figure 4: Session between multiple Instances

4.2.4. Domain

In the figure below, two Domains are represented, where Endpoint A and Endpoint B host Instances belonging to Domain 1, and Endpoint B and Endpoint C host Instances belonging to Domain 2. Instances from the same Domain communicate through a Session. A Session Identifier, or Session ID, may be used as a Discriminator to route the Datagrams to the correct Instance (e.g., to distinguish between the two Instances of Endpoint B), and/or for management purpose.

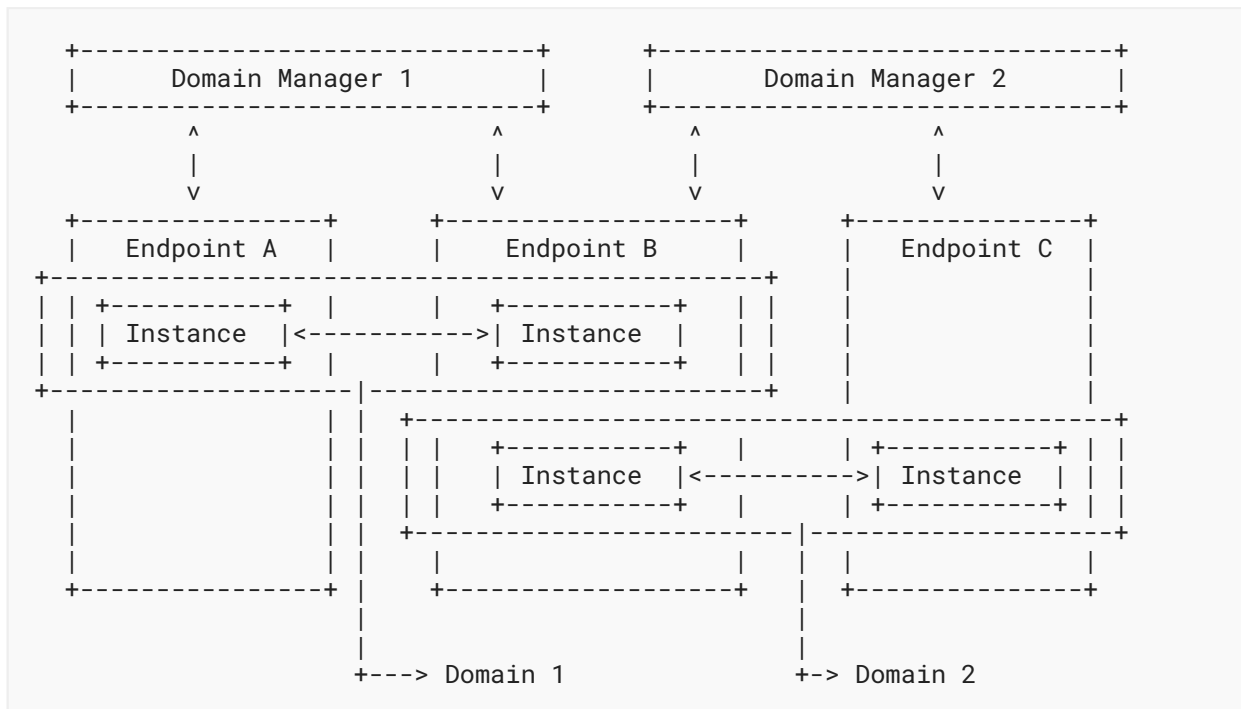


Figure 5: Overview of multiple Domains

4.2.5. Datagram Format

A Datagram is the unit exchanged between SCHC Instances.

It provides a unified representation for:

- compressed packets
- fragmented messages (fragments, acknowledgements, acknowledgement requests, ...)

A Datagram is composed of:

- a RuleID
- the result of the SCHC operation (residue, fragment)
- the Payload

The result of the SCHC operation and the Payload may be empty. As stated in [\[RFC8724\]](#), a Datagram resulting from a compression operation is called a SCHC Packet, and a Datagram resulting from a fragmentation operation is called a Fragment.

```
+-----+-----+-----+
| RuleID | Residue, Fragment, ... | Payload |
+-----+-----+-----+
```

Figure 6: Datagram Format

4.2.5.1. Control Header for Advanced Use Cases

In some deployments, it may be necessary to add information to the Datagram so that it can be properly routed to the correct Instance.

This information may be contained in a specific control structure, external to the Datagram, such as the header of an underlying transport protocol.

This structure may be used to carry various types of information in order to provide functionalities such as:

- Multiplexing (Session, Instance, Context Identifier)
- Protection (Integrity)
- Metadata (Retain information that is lost when performing the SCHC operation, e.g., save the initial value of the EtherType field when it is changed to EtherType=SCHC)

Example representation:

```

+-----+-----+-----+
| Instance ID | CRC | SCHC Datagram |
+-----+-----+-----+

```

Figure 7: SCHC Datagram encapsulated into a transport protocol

Important notice: The structure or transport protocol used to carry these control informations must be a standard format and/or protocol in order to ensure interoperability. Therefore, it might be desirable to design a new protocol tailored to the needs of SCHC, such as a lightweight version of UDP. Various header formats would then be defined to support the aforementioned functions.

5. Deployment models

In this section, we review the deployment scenarios considered in the SCHC Working Group and match the terminology with the scenario characteristics.

5.1. LPWAN deployment

This section considers a typical LPWAN deployment where an IoT device communicates with a gateway or server using SCHC for header compression and decompression. The Instance's Stratum spans [IPv6, UDP, CoAP] --- meaning the Rules in its Context address headers across those three protocol layers. The Carrier Layer (LPWAN link) adjacent below the Stratum's lower boundary carries the Discriminator in a frame field such as the LoRaWAN frame port (fPort).

In this setup, each device features a single SCHC Instance in a single Endpoint. Each Instance is pre-configured with a static Context and Instance configuration. The Discriminator is an explicit field in the LPWAN frame (e.g., fPort) and the Dispatcher is hardcoded in the network stack.

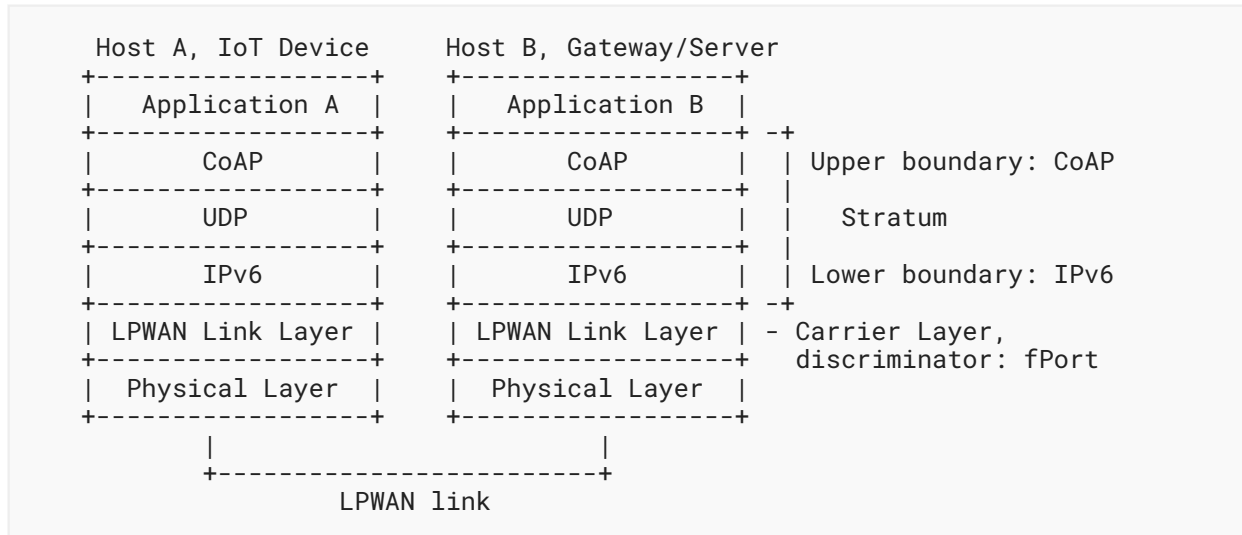


Figure 8: LPWAN deployment

5.2. 6Lo deployments

This section considers deployment scenarios as defined in [I-D.ietf-6lo-schc-15dot4]. The terminology is updated to reflect this of the current document.

In a first deployment, a 6Lo Node (6LN) in a SCHC-Lo network communicates with an external host on the Internet. In this scenario, the 6LN is in direct communication with the 6Lo Border Router (6LBR). SCHC datagrams are only carried over the SCHC-Lo network, between the 6LN. The CoAP header and payload are sent in clear (no encryption), and we assume that the SCHC datagram is small enough to fit in a single 802.15.4 frame.

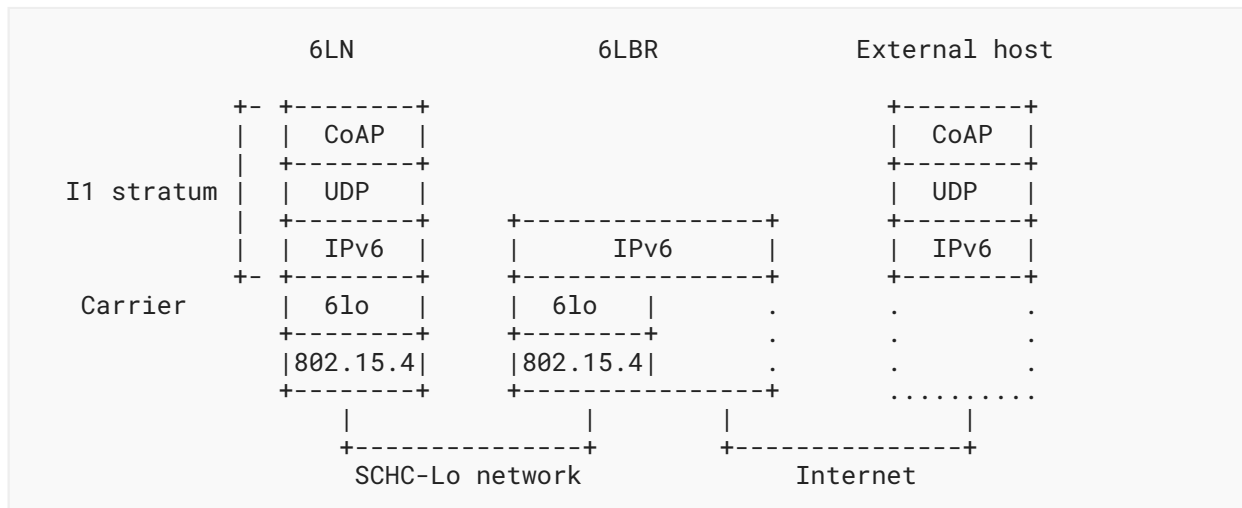


Figure 9: 6Lo deployment: 6Lo Node connected to an external host through a 6Lo Border Router

In this scenario, the 6LN endpoint features a single Instance I1 that compresses the IPv6, UDP and CoAP stratum. The Carrier Layer is the 6Lo adaptation layer which is materialized in frame by the dispatch byte.

A datagram compressed by SCHC results in the following frame format:

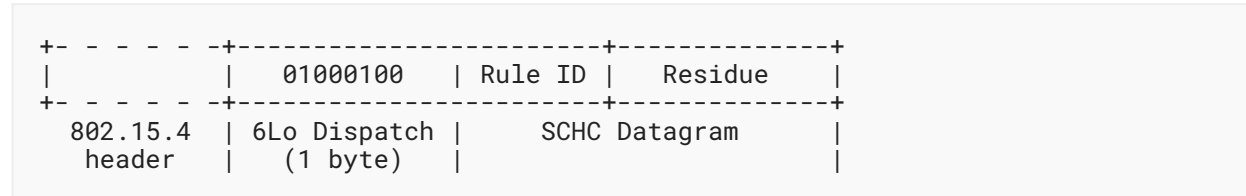


Figure 10: SCHC-Lo Datagram

In a second deployment, a 6LN communicates with an external host on the Internet. In this scenario, the CoAP header is encrypted using DTLS. The 6LN Endpoint features two Instances I1 and I2. I1 compresses the CoAP header while I2 compresses the UDP and IPv6 headers.

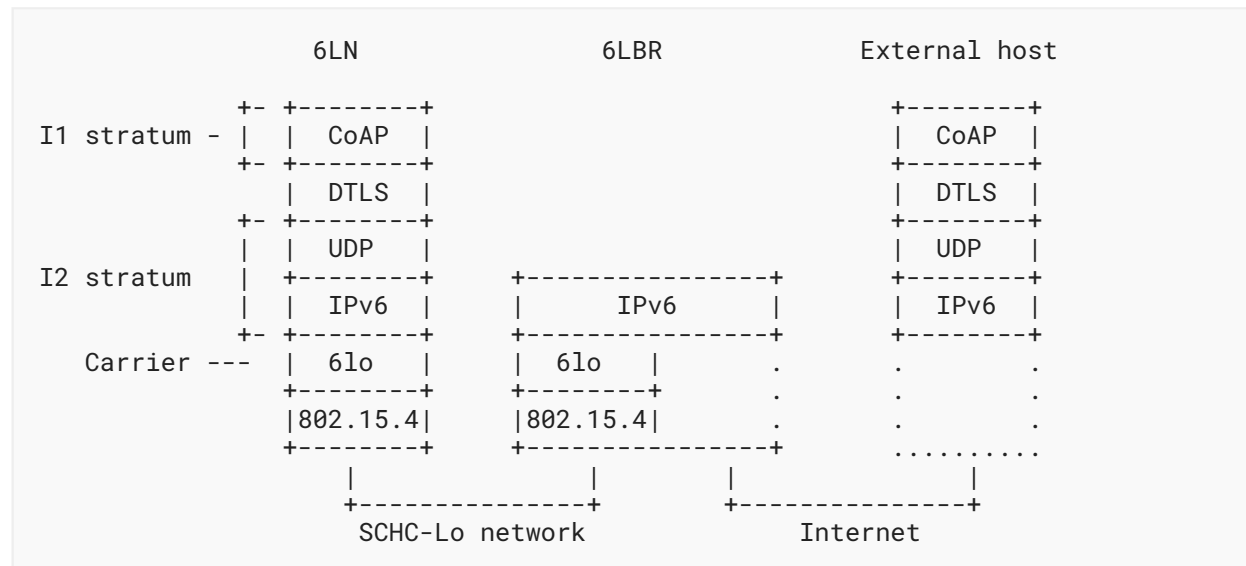


Figure 11: 6Lo deployment: 6Lo Node connected to an external host through a 6Lo Border Router. The CoAP is encrypted using DTLS

The 6Lo Node endpoint features two Instances I1 and I2. I1 corresponding Instance lives on the external host while I2 corresponding Instance lives on the 6LBR.

When the 6LN's CoAP application generates a datagram, that is a CoAP header and eventually a payload, The Dispatcher dispatches the datagram to Instance I1. I1 builds a SCHC Datagram SD1 comprising a Rule ID and SCHC Residue, result of the compression of the CoAP header and Payload. This SCHC Datagram is passed on to the DTLS layer which encrypts the SCHC Residue

and then to UDP and IPv6. Once the IPv6 header is formed, the Dispatcher dispatches the datagram to Instance I2 which compresses the IPv6 and UDP header to form a SCHC Datagram SD2.

Supposing a minimal size DTLS unified header (1byte), the 802.15.4 frame that is transmitted on the SCHC-Lo network is structured as illustrated below.

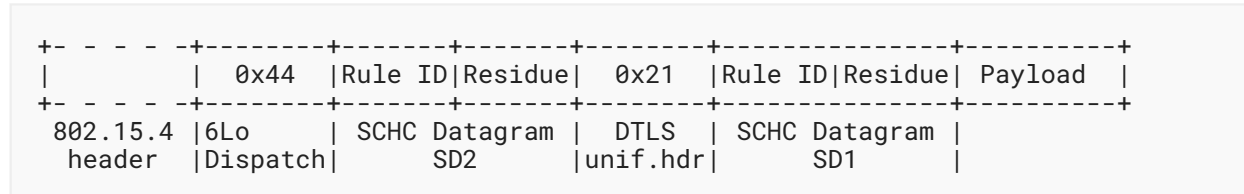


Figure 12: SCHC-Lo Datagram with DTLS

When the 6LBR receives a 802.15.4 frame featuring the 6Lo Dispatch for SCHC, the Dispatcher passe the frame to instance I2 relative on the 6LBR Endpoint. This Instance reconstructs the IPv6 and UDP headers that was compressed by I1. Once decompressed, the frame is reinjected in the stack and further process by the IPv6 layer. The 6LBR then routes the IPv6 frame towards its destination, the external host.

The external host receives a frame structured as follows:

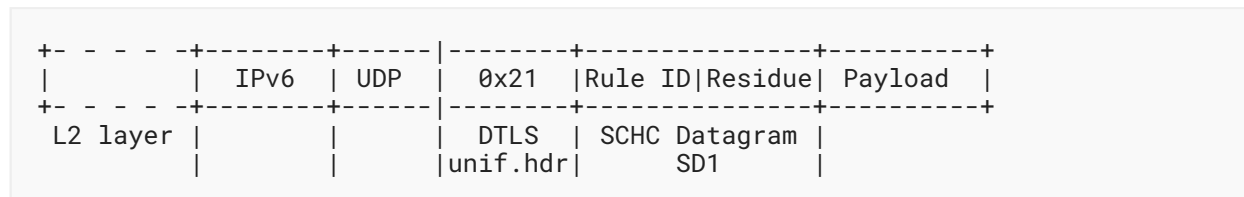


Figure 13: Internet Datagram with DTLS

Once the Carrier Layer has finished processing the ingress datagram, the Dispatcher dispatches the SD1 SCHC Datagram to the corresponding instance, which reconstructs the CoAP header. The CoAP header is then passed by the Dispatcher to the CoAP layer.

6. Operational considerations

Management, Data Models, SCHC Endpoint lifecycle (provisioning, context synchronization), Interoperability, Error handling, etc.

7. Security Considerations

For an architecture document, the security section must analyze:

- Risks specific to SCHC architecture:
 - Compromise or corruption of rules (SoR) and impact on the integrity/ confidentiality of flows.
 - Synchronization drift between SoR/SoV and risk of denial of service (dropped or misrouted packets).
 - Attacks on the Instance Manager (e.g., injection of malicious rules).
 - Interactions with the security mechanisms of the underlying layers (PPP, Ethernet, IPsec, OSCORE, etc.).
- High-level mitigation measures expected in this architecture:
 - Authentication and authorization of entities that manage rules.
 - Logging/auditing of SoR changes.
 - Restoration and rollback of incorrect SCHC profiles.

8. IANA Considerations

This document has no IANA actions.

9. References

9.1. Normative References

- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/rfc/rfc2119>>.
- [RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <<https://www.rfc-editor.org/rfc/rfc8174>>.
- [RFC8724] Minaburo, A., Toutain, L., Gomez, C., Barthel, D., and J.C. Zuniga, "SCHC: Generic Framework for Static Context Header Compression and Fragmentation", RFC 8724, DOI 10.17487/RFC8724, April 2020, <<https://www.rfc-editor.org/rfc/rfc8724>>.
- [RFC8824] Minaburo, A., Toutain, L., and R. Andreasen, "Static Context Header Compression (SCHC) for the Constrained Application Protocol (CoAP)", RFC 8824, DOI 10.17487/RFC8824, June 2021, <<https://www.rfc-editor.org/rfc/rfc8824>>.

9.2. Informative References

- [DRAFT-SCHCLET] Pelov, A., Lampin, Q., and M. Dumay, "SCHClet - Modular Use of the SCHC Framework", Work in Progress, Internet-Draft, draft-ietf-schc-schclet-00, 30 January 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-schc-schclet-00>>.
- [I-D.ietf-6lo-schc-15dot4] Gomez, C. and A. Minaburo, "Transmission of SCHC-compressed packets over IEEE 802.15.4 networks", Work in Progress, Internet-Draft, draft-ietf-6lo-schc-15dot4-12, 20 February 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-6lo-schc-15dot4-12>>.
- [I-D.ietf-core-comi] Veillette, M., Van der Stok, P., Pelov, A., Bierman, A., and C. Bormann, "CoAP Management Interface (CORECONF)", Work in Progress, Internet-Draft, draft-ietf-core-comi-21, 2 March 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-core-comi-21>>.
- [I-D.ietf-intarea-schc-protocol-numbers] Moskowitz, R., Card, S. W., Wiethuechter, A., and P. Thubert, "Protocol Numbers for SCHC", Work in Progress, Internet-Draft, draft-ietf-intarea-schc-protocol-numbers-02, 8 April 2024, <<https://datatracker.ietf.org/doc/html/draft-ietf-intarea-schc-protocol-numbers-02>>.
- [I-D.ietf-schc-access-control] Minaburo, A., Toutain, L., and I. Martinez, "SCHC Access Control", Work in Progress, Internet-Draft, draft-ietf-schc-access-control-00, 13 December 2023, <<https://datatracker.ietf.org/doc/html/draft-ietf-schc-access-control-00>>.
- [I-D.ietf-schc-over-ppp] Thubert, P., "SCHC over PPP", Work in Progress, Internet-Draft, draft-ietf-schc-over-ppp-00, 25 July 2023, <<https://datatracker.ietf.org/doc/html/draft-ietf-schc-over-ppp-00>>.
- [RFC7950] Bjorklund, M., Ed., "The YANG 1.1 Data Modeling Language", RFC 7950, DOI 10.17487/RFC7950, August 2016, <<https://www.rfc-editor.org/rfc/rfc7950>>.
- [RFC8376] Farrell, S., Ed., "Low-Power Wide Area Network (LPWAN) Overview", RFC 8376, DOI 10.17487/RFC8376, May 2018, <<https://www.rfc-editor.org/rfc/rfc8376>>.
- [RFC9011] Gimenez, O., Ed. and I. Petrov, Ed., "Static Context Header Compression and Fragmentation (SCHC) over LoRaWAN", RFC 9011, DOI 10.17487/RFC9011, April 2021, <<https://www.rfc-editor.org/rfc/rfc9011>>.
- [RFC9363] Minaburo, A. and L. Toutain, "A YANG Data Model for Static Context Header Compression (SCHC)", RFC 9363, DOI 10.17487/RFC9363, March 2023, <<https://www.rfc-editor.org/rfc/rfc9363>>.
- [RFC9442] Zuniga, J., Gomez, C., Aguilar, S., Toutain, L., Cespedes, S., Wistuba, D., and J. Boite, "Static Context Header Compression (SCHC) over Sigfox Low-Power Wide Area Network (LPWAN)", RFC 9442, DOI 10.17487/RFC9442, July 2023, <<https://www.rfc-editor.org/rfc/rfc9442>>.

Appendix A. Acknowledgments {numbered="false"}

The authors would like to thank (in alphabetic order): Laurent Toutain.

Authors' Addresses

Alexander Pelov

IMT Atlantique
rue de la Chataigneraie
35576 Cesson-Sevigne Cedex
France
Email: alexander.pelov@imt-atlantique.fr

Pascal Thubert

06330 Roquefort les Pins
France
Email: pascal.thubert@gmail.com

Ana Minaburo

Consultant
35510 Cesson-Sevigne Cedex
France
Email: anaminaburo@gmail.com

Quentin Lampin

Orange
Orange 3 Massifs - 22 Chemin du Vieux Chene
38240 Meylan
France
Email: quentin.lampin@orange.com

Marion Dumay

Orange
Orange 3 Massifs - 22 Chemin du Vieux Chene
38240 Meylan
France
Email: marion.dumay@orange.com